



Mitigating synchronization bottlenecks in high-performance actor-model-based software

Kyle Klenk¹, Mohammad Mahdi Moayeri¹, Junwei Guo²,
Martyn Clark², and Raymond J. Spiteri¹

¹Department of Computer Science, University of Saskatchewan

²Department of Civil Engineering, University of Calgary

PAW-ATM 2024
Atlanta, GA
November 17, 2024

Acknowledgements



R. Spiteri



M.M. Moayeri



M.P. Clark



J. Guo

Motivation

- Default implementations of the actor model can struggle with effective synchronization
- We propose a **batching design strategy** to **improve synchronization performance** and **reduce contention** on specific actors.

Outline

- 1 Background
- 2 SUMMA-Actors with Batching
- 3 Experiments
- 4 Results

Data Assimilation

- Data assimilation is the common practice in simulation science of incorporating observational data into computational models in order to improve their predictive power
 - Bulk Synchronous Parallel (BSP)
 - Fork-Join

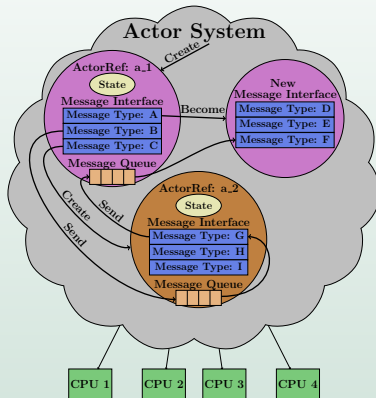
SUMMA

SUMMA simulates the following processes (left), and decomposes the spatial domain into Grouped Response Units (GRUs) and Hydrological Response Units (HRUs) (right)



Clark, M. P., et al. (2015), A unified approach for process-based hydrologic modeling: 1. Modeling concept, *Water Resour. Res.*, 51, 2498-2514.

Actor Model

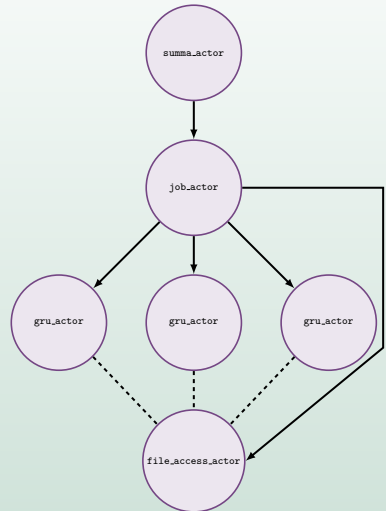


- Private state
- Communicate only through messages
- Three primitive operations
 - Create
 - Send
 - Become

SUMMA-Actors

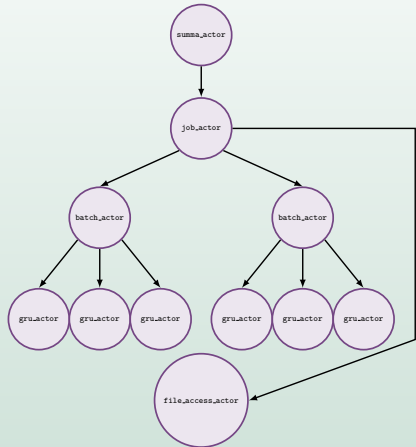
- Based on the C++ Actor Framework (CAF)
- Each GRU is its own actor
- Increases the efficiency and fault-tolerance of SUMMA
- **This design bottlenecked performing data assimilation**

Klenk, K., Spiteri, R.J. Improving resource utilization and fault tolerance in large simulations via actors. Cluster Comput. (2024).

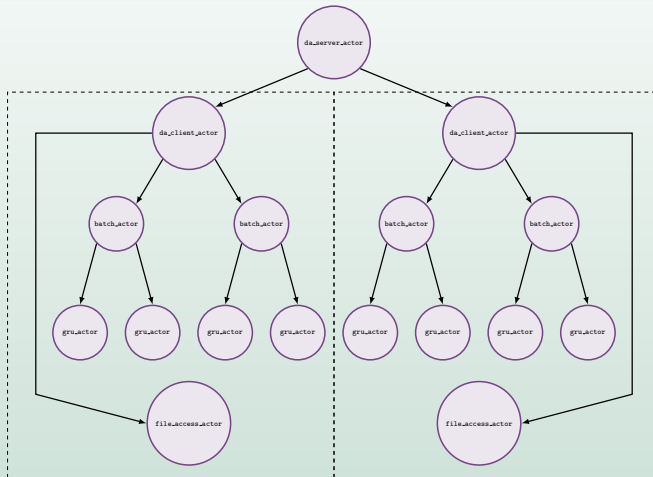


SUMMA-Actors in Data-Assimilation Mode for a Shared-Memory Environment

- **Problem:** Contention on the `job_actor`
- **Solution:** Introduce a layer of `batch_actor`



SUMMA-Actors in Data-Assimilation Mode for a Distributed-Memory Environment



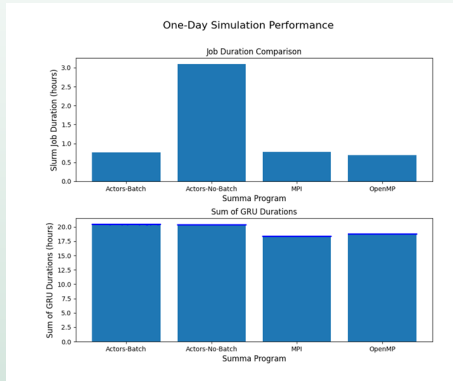
Test cases

- E1: one-day simulation (24 hourly data windows) executed on a single node
- E2: one-week simulation (168 hourly data windows) executed on a single node
- E3: one-month simulation (744 hourly data windows) executed on a single node
- E4: one-month simulation (744 hourly data windows) executed on 10, 20, and 40 nodes

Configurations of SUMMA

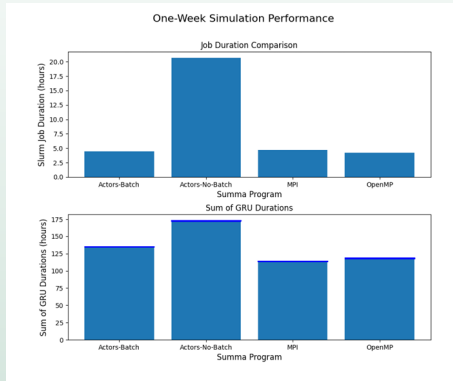
- 517,315 GRUs (North America) with forcing data from January 2019
- Configurations of SUMMA for E1, E2, and E3:
 - SUMMA-Actors in data-assimilation mode with batching
 - SUMMA-Actors in data-assimilation mode with no batching
 - SUMMA-MPI
 - SUMMA-OpenMP
- Configurations of SUMMA for E4:
 - SUMMA-Actors in data-assimilation mode with batching
 - SUMMA-MPI
 - SUMMA-MPI+OpenMP

Experiment E1: One-day Simulation, Single Node



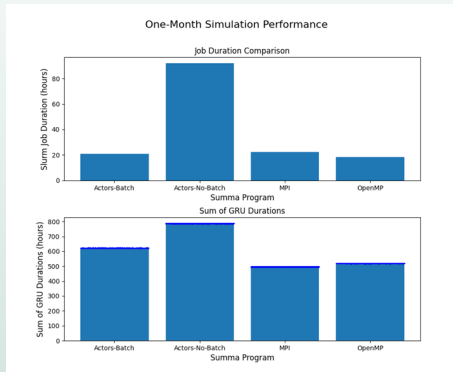
- SUMMA-OpenMP is **9.6% faster** than SUMMA-Actors with batching
- SUMMA-Actors with batching is **2.5% faster** than SUMMA-MPI
- SUMMA-Actors has some overhead due to using Fortran and C++ (bottom figure)

Experiment E2: One-week Simulation, Single Node



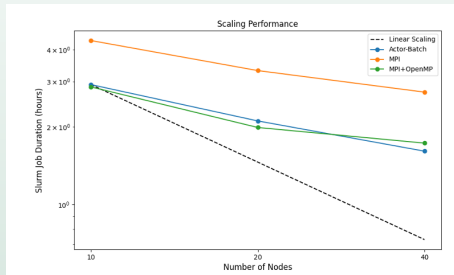
- SUMMA-OpenMP is **6.7% faster** than SUMMA-Actors with batching
- SUMMA-Actors with batching is **5.6% faster** than SUMMA-MPI

Experiment E3: One-month Simulation, Single Node



- SUMMA-OpenMP is **13.3% faster** than SUMMA-Actors with batching
- SUMMA-Actors with batching is **6.1% faster** than SUMMA-MPI

Experiment E4: One-month Simulation, Multi Node



- SUMMA-Actors and SUMMA-MPI+OpenMP outperformed SUMMA-MPI
- SUMMA-Actors outperforms SUMMA-MPI+OpenMP in the 40 node case

Conclusions

- We proposed an actor-based batching strategy that buffers communication
- Experiments demonstrate the batching strategy significantly improves performance
- The strategy outperforms MPI and is competitive with OpenMP

Thank You

- Contact:
 - Kyle Klenk: kyle.klenk@usask.ca
 - Mohammad Mahdi Moayeri: mahdi.moayeri@usask.ca
 - Junwei Guo: junwei.guo@ucalgary.ca
 - Martyn P. Clark: martyn.clark@ucalgary.ca
 - Raymond J. Spiteri: spiteri@cs.usask.ca